

Redes Neuronales Recurrentes Análogas con Pesos Racionales

Andrés Sicard Ramírez
asicard@eafit.edu.co

Juan C. Agudelo Agudelo
jagudelo@eafit.edu.co

Mario E. Vélez Ruiz
mvelez@eafit.edu.co

Grupo de Lógica y Computación
Escuela de Ciencias y Humanidades
Universidad EAFIT; Medellín, Colombia

Resumen

Se definen las redes neuronales recurrentes análogas (ARNNs) y las funciones recursivas y se demuestra la equivalencia computacional entre las ARNNs con pesos racionales y las máquinas de Turing por medio de la equivalencia computacional entre las ARNNs con pesos racionales y las funciones recursivas.

Abstract

We define analog recurrent neural networks (ARNN) and recursive functions then we demonstrate computational equivalence between ARNNs with rational weights and Turing machines mediating computational equivalence between ARNNs with rational weights and recursive functions.

1 Redes Neuronales Recurrentes Análogas

Definición 1 (Red Neuronal Recurrente Análoga). *Una ARNN (Analog Recurrent Neural Network) [11] es una red neuronal compuesta por N procesadores elementales llamados neuronas, cada neurona tiene asociado un valor de activación $x_i(t)$, en cada instante discreto de tiempo t la red neuronal recibe M entradas binarias $u_j(t)$. La dinámica de la red consiste en calcular los valores de activación de las neuronas de acuerdo a las entradas y los valores de activación de las neuronas en el instante de tiempo anterior. Cada neurona calcula su valor de activación de acuerdo a la ecuación:*

$$x_i(t+1) = \sigma \left(\left(\sum_{j=1}^N a_{ij} \cdot x_j(t) \right) + \left(\sum_{j=1}^M b_{ij} \cdot u_j(t) \right) + c_i \right), \quad (1)$$

donde a_{ij} es el peso del enlace entre la neurona x_j y la neurona x_i , b_{ij} es el peso del enlace entre la neurona u_j y la neurona x_i y c_i es el peso constante asociado a la neurona x_i . La función σ es llamada “función de activación” o “función de respuesta” y el argumento que recibe es una función de las entradas a la neurona llamada “función de red”. En cada instante de tiempo la salida de la red es el valor de activación de un subconjunto de ℓ neuronas $y_1(t), \dots, y_\ell(t)$.

Los valores de activación de las neuronas son representados por un vector $\mathbf{X}(t)$ de dimensión $N \times 1$, las entradas por un vector $\mathbf{U}(t)$ de dimensión $M \times 1$, los pesos de los enlaces entre las neuronas por una matriz $\mathbf{A}$ de dimensión $N \times N$, los pesos de los enlaces entre las entradas y las neuronas por una matriz

B de dimensión $N \times M$ y los pesos constantes por un vector C de dimensión $N \times 1$. De esta forma se representa la dinámica total de la red por la ecuación matricial:

$$\mathbf{X}(t+1) = \sigma(\mathbf{A} \cdot \mathbf{X}(t) + \mathbf{B} \cdot \mathbf{U}(t) + \mathbf{C}). \quad (2)$$

Definición 2 (Computación en una ARNN). Una computación en una ARNN es una sucesión de cálculos en instantes discretos de tiempo de los valores de activación $\mathbf{X}(t+1)$ de las neuronas de acuerdo a los valores de activación $\mathbf{X}(t)$ y las entradas $\mathbf{U}(t)$ en el instante de tiempo anterior, teniendo en cuenta los enlaces y pesos de la red $\mathbf{A}$, $\mathbf{B}$ y $\mathbf{C}$, de acuerdo a la ecuación (2). Las salidas en cada iteración son un subconjunto de los valores de activación calculados $y_1(t), \dots, y_\ell(t)$. Al inicio de la computación, tiempo $t=0$, cada neurona tiene un valor de activación inicial $x_i(0)$ (normalmente $x_i(0) = 0$ para $i = 1, 2, \dots, N$). La computación termina cuando no hay más entradas.

Hay que tener en cuenta que en la mayoría de ARNNs se introduce un retardo r en la computación, es decir, los primeros r valores de activación de las neuronas de salida no deben ser tomados en cuenta como parte del cómputo, esto se debe a que las entradas por lo general no llegan directamente a las neuronas de salida y para llegar a afectarlas tienen primero que propagarse por otras neuronas de la red, este tiempo de retardo depende de la estructura específica de la red.

2 Funciones Recursivas

Definición 3 (Función Numérico-Teórica). Se denomina función numérico-teórica a toda función definida en k -tuplas de números naturales $\mathbb{N}^k$ y evaluada en números naturales $\mathbb{N} = \{0, 1, 2, \dots\}$ [6], es decir,

$$f \text{ es una función numérico-teórica} \iff f: \mathbb{N}^k \rightarrow \mathbb{N}.$$

Definición 4 (Representación k -tuplas). Para efectos de simplificación en la notación se utilizará el símbolo $\vec{x}_k$ para representar la tupla $x_1, x_2, \dots, x_k$.

Definición 5 (Función Recursiva). Las funciones recursivas son un subconjunto de las funciones numérico-teóricas, se dice que una función $f: \mathbb{N}^k \rightarrow \mathbb{N}$ es recursiva si [8, 9, 3, 6]¹:

1. Es una de las siguientes funciones:

- $Z(x) = 0$, función constante cero.
- $S(x) = x + 1$, función sucesor.
- $P_k^n(\vec{x}_n) = x_k$, función k -ésima proyección en n variables, $k \leq n$.

Estas funciones son llamadas funciones recursivas básicas y son los axiomas de la teoría.

2. Puede obtenerse por sucesión finita de aplicaciones de las siguientes reglas:

- Esquema de composición: Si $f(\vec{y}_n)$ y $g_1(\vec{x}_k), \dots, g_n(\vec{x}_k)$ son funciones recursivas entonces $h(\vec{x}_k) = f(g_1(\vec{x}_k), \dots, g_n(\vec{x}_k))$ es una función recursiva.

¹En [5] se da una presentación alternativa de las funciones recursivas

- *Esquema de recurrencia primitiva: Si $f(\vec{x}_k)$ y $g(\vec{x}_k, y, z)$ son funciones recursivas entonces $h(\vec{x}_k, y)$ definida por:*

$$\begin{aligned} h(\vec{x}_k, 0) &= f(\vec{x}_k), \\ h(\vec{x}_k, y + 1) &= g(\vec{x}_k, y, h(\vec{x}_k, y)), \end{aligned}$$

es una función recursiva.

- *Esquema de minimización: Si $f(\vec{x}_k, y)$ es una función recursiva, se puede construir la función $h(\vec{x}_k) = \mu_y(f(\vec{x}_k, y) = 0)$ donde:*

$$\mu_y(f(\vec{x}_k, y) = 0) = \begin{cases} \text{El menor } y \text{ tal que } f(\vec{x}_k, z) \text{ esté definida para todo } z \leq y \\ \text{y } f(\vec{x}_k, y) = 0, \\ \text{No está definida si no existe tal } y. \end{cases}$$

Si la función $\mu_y(f(\vec{x}_k, y) = 0)$ está definida para todo $\vec{x}_k$ entonces la función $h(\vec{x}_k)$ es una función recursiva total, de otro modo, la función $h(\vec{x}_k)$ es una función recursiva parcial.

Es bien conocida la equivalencia entre las funciones recursivas parciales y las funciones que se pueden computar en una máquina de Turing (funciones Turing-computables), es decir, toda función Turing-computable puede ser expresada como una función recursiva parcial y para toda función recursiva parcial se puede construir una máquina de Turing que la compute. Esta equivalencia es presentada en [5, 9, 2, 3, 6].

3 Relación entre ARNN y Máquina de Turing

Al restringir algunos parámetros de las ARNNs, tales como los posibles valores que pueden tomar los pesos y las características de la función de activación, se puede obtener modelos con diferente poder de computación. En particular, si se tiene una ARNN con función de activación σ definida por:

$$\sigma(x) = \begin{cases} 0 & \text{si } x < 0, \\ x & \text{si } 0 \leq x \leq 1, \\ 1 & \text{si } x > 1, \end{cases} \quad (3)$$

y se restringen los posibles valores que pueden tomar los pesos (a_{ij} , b_{ij} y c_i) de la red a los números racionales, los valores de activación de las neuronas serán valores racionales en el intervalo $[0, 1]$ y el poder de computación de la ARNN es equivalente al de una máquina de Turing. Para demostrar tal equivalencia se aprovechará la equivalencia entre las funciones recursivas parciales y las funciones Turing-computables, se demostrará la equivalencia computacional entre las ARNNs con pesos racionales y las funciones recursivas parciales y a partir de este hecho se aceptará la equivalencia computacional entre las ARNNs con pesos racionales y las máquinas de Turing.

Antes de demostrar la equivalencia computacional entre las ARNNs con pesos racionales y las funciones recursivas parciales se presentarán algunas definiciones:

Definición 6 (Función i-ésimo Primo). *Se denotará por $Pn(i)$ a la función que calcula el i-ésimo número primo. Se puede demostrar que esta función es recursiva.*

Definición 7 (Codificación de Gödel). Sea $\Sigma = \{a_1, a_2, a_3, \dots\}$ un alfabeto enumerable, y Σ^* el conjunto de las palabras construibles sobre Σ . La codificación de Gödel permite dar una codificación numérica $NG: \Sigma^* \rightarrow \mathbb{N}$ a las palabras de Σ^* , esta codificación se basa en el teorema fundamental de la aritmética y se obtiene aplicando los siguientes pasos:

1. A cada símbolo del alfabeto Σ se le asigna un número natural, $N(a_i) = i$ para todo $a_i \in \Sigma$, $i \in \mathbb{N}$.
2. El número de Gödel de una palabra $\alpha = s_1 s_2 \dots s_n \in \Sigma^*$ está dado por:

$$NG(\alpha) = 2^{N(s_1)} \cdot 3^{N(s_2)} \dots P_n(n)^{N(s_n)} = \prod_{i=1}^n P_n(i)^{N(s_i)}. \quad (4)$$

Definición 8 (Codificación de k-tuplas de Naturales en Naturales). Para codificar k-tuplas de naturales en naturales se utilizará la codificación de Gödel de la siguiente manera: Sea $(x_1, \dots, x_k) \in \mathbb{N}^k$ entonces definimos $CK: \mathbb{N}^k \rightarrow \mathbb{N}$ por:

$$CK(x_1, \dots, x_k) = 2^{x_1} \cdot 3^{x_2} \dots P_n(k)^{x_k} = \prod_{i=1}^k P_n(i)^{x_i}. \quad (5)$$

Se puede demostrar que CK es una función recursiva.

Definición 9 (Decodificación de k-tuplas de Naturales). Se puede obtener los elementos de las k-tuplas codificadas aplicando la codificación anterior (ec. 5) por:

$$E_i(y) = \text{máximo } w \leq y \text{ tal que } P_n(i)^w \text{ divide a } y; w, y \in \mathbb{N}. \quad (6)$$

De este modo $E_i(2^{x_1} \cdot 3^{x_2} \dots P_n(i)^{x_i} \dots P_n(n)^{x_n}) = x_i$. Se puede demostrar que la función E_i es una función recursiva.

Teorema 1. La computación de toda ARNN con pesos racionales puede ser expresada como una función recursiva.

Demostración. La demostración se hace para $a_{ij}, b_{ij}, c_i \in \mathbb{Q}^+ \cup \{0\}$.

1. Se expresará primero la función que calcula el valor de activación para una neurona (ec. 1) como una función recursiva, esta ecuación se puede ver como una función f_i de la forma $f_i: \mathbb{Q}^N \times \{0, 1\}^M \rightarrow \mathbb{Q}$ donde:

$$f_i(\mathbf{X}(t), \mathbf{U}(t)) = \sigma \left(\left(\sum_{j=1}^N a_{ij} \cdot x_j(t) \right) + \left(\sum_{j=1}^M b_{ij} \cdot u_j(t) \right) + c_i \right), \text{ con :} \quad (7)$$

$\mathbf{X}(t) = x_1(t), \dots, x_n(t); 0 \leq x_i(t) \leq 1 \in \mathbb{Q}$, vector de activación.

$\mathbf{U}(t) = u_1(t), \dots, u_m(t); u_i(t) \in \{0, 1\}$, vector de entradas.

$a_{ij}, b_{ij}, c_i \in \mathbb{Q}^+ \cup \{0\}$, pesos de la red.

Teniendo en cuenta que la teoría de funciones recursivas está definida sobre funciones numérico-teóricas, para expresar f_i como una función recursiva se llevará f_i a la función f'_i de la forma $f'_i: \mathbb{N}^{2N+M} \rightarrow \mathbb{N}$, dicha función calculará el valor de activación de una neurona codificando el resultado en los números naturales mediante la función CK (ec. 5).

2. El argumento de la función σ (ec. 7), llamado función de red, puede ser visto como la función net_i de la forma $net_i: \mathbb{Q}^N \times \{0, 1\}^M \rightarrow \mathbb{Q}$ donde:

$$net_i(\mathbf{X}(t), \mathbf{U}(t)) = \left(\sum_{j=1}^N a_{ij} \cdot x_j(t) \right) + \left(\sum_{j=1}^M b_{ij} \cdot u_j(t) \right) + c_i. \quad (8)$$

3. Todo número racional positivo $x \in \mathbb{Q}^+$ puede ser expresado como la pareja ordenada de números naturales (Nx, Dx) ; $Nx, Dx \in \mathbb{N}$, Nx simboliza el numerador de x y Dx simboliza el denominador de x . 0 se representa por la pareja $(0, 1)$.
4. Con base en el numeral (3) se redefine la función net_i (ec. 8) por $net'_i: \mathbb{N}^{2N+M} \rightarrow \mathbb{N}^2$ donde:

$$net'_i(\mathbf{X}'(t), \mathbf{U}(t)) = \left(\sum_{j=1}^N \frac{Na_{ij}}{Da_{ij}} \cdot \frac{Nx_j(t)}{Dx_j(t)} \right) + \left(\sum_{j=1}^M \frac{Nb_{ij}}{Db_{ij}} \cdot u_j(t) \right) + \frac{Nc_i}{Dc_i}, \text{ con :} \quad (9)$$

$\mathbf{X}'(t) = Nx_1(t), Dx_1(t), \dots, Nx_n(t), Dx_n(t)$; $Nx_i(t), Dx_i(t) \in \mathbb{N}$, vector de activación expresado en parejas de números naturales.

$\mathbf{U}(t) = u_1(t), \dots, u_m(t)$; $u_i(t) \in \{0, 1\}$, vector de entradas.

$Na_{ij}, Da_{ij}, Nb_{ij}, Db_{ij}, Nc_i, Dc_i \in \mathbb{N}$, pesos de la red expresados en parejas de números naturales.

Por simplificación en la notación se escribirá net'_i en lugar de $net'_i(\mathbf{X}'(t), \mathbf{U}(t))$.

5. La función net'_i se puede partir en dos funciones: $num_i: \mathbb{N}^{2N+M} \rightarrow \mathbb{N}$ y $den_i: \mathbb{N}^{2N+M} \rightarrow \mathbb{N}$, para calcular el primer elemento (numerador) de net'_i y el segundo elemento (denominador) de net'_i respectivamente, es decir, $net'_i = (num_i, den_i)$. den_i y num_i se pueden expresar como funciones recursivas así:

$$den_i(\mathbf{X}'(t), \mathbf{U}(t)) = mcm(Da_{i1} \cdot Dx_1(t), \dots, Da_{in} \cdot Dx_n(t), Db_{i1}, \dots, Db_{im}, Dc_i), \quad (10)$$

donde mcm es la función mínimo común múltiplo, mcm es una función recursiva. Por simplificación en la notación se escribirá den_i en lugar de $den_i(\mathbf{X}'(t), \mathbf{U}(t))$.

$$num_i(\mathbf{X}'(t), \mathbf{U}(t)) = \left(\sum_{j=1}^N \left(qt(Da_{ij} \cdot Dx_j(t), den_i) \cdot (Na_{ij} \cdot Nx_j(t)) \right) \right) + \left(\sum_{j=1}^M \left(qt(Db_{ij}, den_i) \cdot (Nb_{ij} \cdot u_j(t)) \right) \right) + qt(Dc_i, den_i) \cdot Nc_i. \quad (11)$$

donde $qt(x, y)$ es el cociente de dividir y por x , qt es una función recursiva, además, la suma, la multiplicación y den_i son funciones recursivas, por lo tanto num_i es una función recursiva por ser composición de funciones recursivas. Por simplificación en la notación se escribirá num_i en lugar de $num_i(\mathbf{X}'(t), \mathbf{U}(t))$.

6. Se redefine la función σ (ec. 3) por σ' para que opere sobre dos elementos codificando el resultado en los números naturales mediante la función CK (ec. 5):

$$\sigma'(x_1, x_2) = \begin{cases} CK(1, 1) & \text{si } x_1 > x_2, \\ CK(x_1, x_2) & \text{si } x_1 \leq x_2. \end{cases} \quad (12)$$

$\sigma'(x_1, x_2)$ es una función recursiva.

7. Luego f'_i de la forma $f'_i: \mathbb{N}^{2N+M} \rightarrow \mathbb{N}$ donde:

$$f'_i(\mathbf{X}'(t), \mathbf{U}(t)) = \sigma'(net'_i) = \sigma'(num_i, den_i), \quad (13)$$

calcula el valor de activación de una neurona codificando el resultado en los números naturales mediante la función CK (ec. 5) y es una función recursiva.

8. Teniendo ya expresada la función de activación para una neurona como una función recursiva, se expresará la función de activación o de dinámica de la red para todas las neuronas como una función recursiva, dicha función se puede ver como una función de la forma $f: \mathbb{Q}^N \times \{0, 1\}^M \rightarrow \mathbb{Q}^N$. Para definir f con base en f'_i se redefine f por f' de la forma $f': \mathbb{N}^{2N+M} \rightarrow \mathbb{N}^{2N}$ donde:

$$f'(\mathbf{X}'(t), \mathbf{U}(t)) = (E_1(f'_1), E_2(f'_1), \dots, E_1(f'_n), E_2(f'_n)), \quad (14)$$

donde E_i es la función de decodificación dada por la ecuación (6).

Para expresar f' como una función recursiva se redefine f' por f'' de la forma $f'': \mathbb{N}^{2N+M} \rightarrow \mathbb{N}$ donde:

$$f''(\mathbf{X}'(t), \mathbf{U}(t)) = CK\left(f'(\mathbf{X}'(t), \mathbf{U}(t))\right). \quad (15)$$

La función f'' calcula el valor de activación de todas las neuronas y los codifica en los números naturales mediante la función CK (ec. 5). f'' es una función recursiva debido a que f'_i, E_i y CK son funciones recursivas.

9. Para simular el comportamiento de una computación en una ARNN en t instantes discretos de tiempo, se debe realizar una sucesión de t cálculos de los valores de activación de las neuronas, comenzando con valores de activación $x_i(0) = 0$ para todas las neuronas $1 \leq i \leq N$ y recibiendo en cada instante de tiempo t' , $0 \leq t' \leq t-1$, el vector de entradas $\mathbf{U}(t')$. Esto se puede expresar mediante la función:

$$\begin{aligned} cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), 0) &= 2^0 \cdot 3^1 \dots P_n(2n-1)^0 \cdot P_n(2n)^1, \\ cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t+1) &= f''(E_1(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)), \\ &\quad E_2(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)), \dots, \\ &\quad E_{2n-1}(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)), \\ &\quad E_{2n}(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)), \\ &\quad \mathbf{U}(t)). \end{aligned} \quad (16)$$

La función cmp es una función recursiva y simula la computación de una ARNN en t instantes discretos de tiempo, luego la computación de toda ARNN con pesos racionales enteros se puede expresar como una función recursiva.

El valor de activación de la neurona i en el instante de tiempo t se puede calcular aplicando la función:

$$x_i(t) = \frac{E_{2i-1}\left(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)\right)}{E_{2i}\left(cmp(\mathbf{U}(0), \dots, \mathbf{U}(t-1), t)\right)}, \quad (17)$$

aplicando esta función a las neuronas de salida se obtienen los valores de salida de la ARNN en el instante de tiempo t .

□

La demostración se puede extender a $\mathbb{Q} = \mathbb{Q}^- \cup \{0\} \cup \mathbb{Q}^+$ codificando cada racional $x \in \mathbb{Q}$ por la tripleta (Sx, Nx, Dx) , donde $Sx \in \{0, 1\}$ determina el símbolo del número racional ($0 = +, 1 = -$) y $Nx, Dx \in \mathbb{N}$ representan el numerador y denominador respectivamente. El 0 se representa por $(0, 0, 1)$. Además, se debe tener en cuenta el signo en las sumatorias de la siguiente manera:

$$(S_1, N_1, D_1) + (S_2, N_2, D_2) = \begin{cases} (0, N_1 \cdot D_2 + D_1 \cdot N_2, D_1 \cdot D_2) & \text{si } S_1 = S_2 = 0, \\ (1, N_1 \cdot D_2 + D_1 \cdot N_2, D_1 \cdot D_2) & \text{si } S_1 = S_2 = 1, \\ (0, |N_1 \cdot D_2 - D_1 \cdot N_2|, D_1 \cdot D_2) & \text{si } S_1 = 0, S_2 = 1 \text{ y } N_1 \cdot D_2 > D_1 \cdot N_2, \\ & \text{ó } S_1 = 1, S_2 = 0 \text{ y } N_1 \cdot D_2 < D_1 \cdot N_2, \\ (1, |N_1 \cdot D_2 - D_1 \cdot N_2|, D_1 \cdot D_2) & \text{de otro modo.} \end{cases} \quad (18)$$

La definición de computación presentada en [11] se enfoca en redes con un protocolo de entrada-salida en la que se tienen solo dos líneas de entrada $D(t)$ y $V(t)$, $D(t)$ es la entrada por donde se ingresan los datos a computar y $V(t)$ sirve de línea de validación de la entrada indicando cuando $D(t)$ está activa o no. De forma similar, solo se tienen dos líneas de salida, una para la salida de datos y otra para la validación de la salida, denotadas por $G(t)$ y $H(t)$ respectivamente. Teniendo en cuenta este protocolo, para computar funciones parciales de la forma $f: \{0, 1\}^+ \rightarrow \{0, 1\}^+$, donde $+$ representa la cerradura positiva de Kleene [1, 4], se tiene que las entradas a la ARNN para el valor $w = w_1 \dots w_k$ a computar son:

$$D(t) = \begin{cases} w_t & \text{si } 1 \leq t \leq k, \\ 0 & \text{si } t > k, \end{cases} \quad (19)$$

$$V(t) = \begin{cases} 1 & \text{si } 1 \leq t \leq k, \\ 0 & \text{si } t > k. \end{cases} \quad (20)$$

Se dice que la ARNN computa $f(w)$ si existe $r \in \mathbb{N}$ tal que:

$$G(t) = \begin{cases} f(w)_{[t-r+1]} & \text{si } r \leq t \leq r + |f(w)| - 1, \text{ donde } |f(w)| = \text{longitud de } f(w), \\ 0 & \text{de otro modo,} \end{cases} \quad (21)$$

y

$$H(t) = \begin{cases} 1 & \text{si } r \leq t \leq r + |f(w)| - 1, \\ 0 & \text{de otro modo,} \end{cases} \quad (22)$$

Si $H(t) = 0$ para todo t o $H(t) = 1$ para todo $t \geq r$ se dice que $f(w)$ no está definida.

Teniendo en cuenta esta definición de computación se puede redefinir la función *cmp* (ec. 16) con base en las entradas $D(t), V(t)$ y se puede minimizar la función con base en la salida $H(t)$ para obtener r y $r + |f(w)| - 1$, las salidas de la red serán los valores de activación de $G(t)$ para los valores de *cmp* con $r \leq t \leq r + |f(w)| - 1$. Si la ARNN computa $f(w)$ para todo $w \in \{0, 1\}^+$ entonces $r, r + |f(w)| - 1$ y *cmp* serán funciones recursivas totales, si $f(w)$ no está definida para algún $w \in \{0, 1\}^+$ en la ARNN entonces $r, r + |f(w)| - 1$ y *cmp* serán funciones recursivas parciales.

Teorema 2. *Para toda función recursiva existe una ARNN con pesos racionales que la computa.*

La demostración a este teorema es presentada en [10], en esta demostración se construyen las ARNNs con pesos racionales que computan las funciones básicas recursivas, luego se muestra como se pueden obtener los esquemas de composición, recurrencia primitiva y minimización indicando como se deben interconectar las ARNNs que computan las funciones que se suponen recursivas en cada uno de los esquemas. En [11, 12] se hace una demostración alternativa de la equivalencia entre ARNNs con pesos racionales y máquinas de Turing, en esta demostración se prueba que toda *p-stack machine* [7] puede ser simulada por una ARNN con pesos racionales y como las *p-stack machines* con $p \geq 2$ son equivalentes a una máquina de Turing [7] entonces las ARNNs con pesos racionales son equivalentes a las máquinas de Turing, en este trabajo no sólo se demuestra la equivalencia computacional entre los dos modelos sino también la equivalencia en complejidad algorítmica.

4 Conclusiones

El resultado de restringir a los números racionales los valores que pueden tomar los pesos en una ARNN con función de activación σ (ec. 3) es que las neuronas pueden tomar un número infinito contable de valores de activación ($0 \leq x \leq 1 \in \mathbb{Q}$), esto hace que la ARNN pueda estar en un número finito, no acotado de ‘configuraciones’ debido a que el número de neuronas es finito, entendiendo por configuración una combinación de los valores de activación de las neuronas. Esta es una situación similar a la que se presenta en una máquina de Turing, debido a que la cinta de la máquina tiene un número infinito contable de celdas, un número finito de estados y un alfabeto de entrada-salida finito; al entender una configuración en una máquina de Turing como la combinación del número de la celda que se está leyendo, el símbolo que se está leyendo y el estado de la máquina también se tiene que la máquina de Turing puede estar en un número finito no acotado de configuraciones.

El número de configuraciones que puede alcanzar una máquina también puede ser pensada como su capacidad de memoria, desde este punto de vista tanto las ARNNs con pesos racionales como las máquinas de Turing tienen una capacidad no acotada de memoria.

Agradecimientos

Este artículo fue financiado por la universidad EAFIT, bajo el proyecto de investigación número 817424.

Referencias

- [1] ALFRED V. AHO, RAVI SETHI Y JEFFREY D. ULLMAN. “Compiladores: principios, técnicas y herramientas”. Wilmington, Delaware: Addison-Wesley Iberoamericana (1990).
- [2] GEORGE BOOLOS Y RICHARD JEFFREY. “Computability and logic”. Cambridge University Press, 3rd edición (1989).
- [3] XAVIER CAICEDO. “Elementos de lógica y calculabilidad”. Santafé de Bogotá: Una empresa docente, U. de los Andes, 2a edición (1990).
- [4] DECOROSO CRESPO. “Informática teórica. Primera parte”. Madrid: Departamento de Publicaciones de la Facultad de Informática, U. Politécnica de Madrid. (1983).

- [5] MARTIN DAVIS. “Computability and unsolvability”. New York: Dover Publications, Inc. (1982).
- [6] RAÚL GÓMEZ MARÍN Y ANDRÉS SICARD RAMÍREZ. “Informática teórica: Elementos propedeúticos”. Medellín: Fondo Editorial U. EAFIT (2001).
- [7] JOHN HOPCROFT Y JEFFEREY ULLMAN. “Introducción a la teoría de autómatas, lenguajes y computación”. México D.F.: Compañía Editorial Continental, S.A. (CECSA) (1997).
- [8] STEPHEN C. KLEENE. “Introducción a la metamatemática”. Colección: Estructura y Función. Madrid: Editorial Tecnos (1974).
- [9] MARVIN L. MINSKY. “Computation: finite and infinite machines”. Englewood Cliffs, N.J.: Prentice-Hall, Inc. (1967).
- [10] J. P. NETO, H. T. SIEGELMANN, J. F. COSTA Y C. P. ARAUJO. Turing universality of neural nets (revisited). Publicado en: Lecture notes in Computer Science: computer aided system theory. Edited by F. Pichler and R. Moreno Diaz. Eprint: www.cs.math.ist.utl.pt/cgi-bin/uncgi/bib2html.tcl?author=fgc (1977).
- [11] HAVA T. SIEGELMANN. “Neural networks and analog computation. Beyond the Turing limit”. Progress in Theoretical Computer Science. Boston: Birkhäuser (1999).
- [12] HAVA T. SIEGELMANN Y EDUARDO D. SONTAG. On computational power of neural networks. *Journal of Computer and System Sciences* **50(1)**, 132–150 (1995).